

Software Lifecycle Management



Fachbereich Informatik und Automation (IAT)

Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Einführung

Agenda

- Vorstellung und Organisatorisches
- Literaturhinweise
- "Was ist Software?"
- Software-Entwicklung in der realen Welt
- Vorlesungsinhalte und Abgrenzungen

Organisatorisches (allgemein)

Modul	Software Lifecycle Management
Fachsemester	4. Semester
Dozent	Dr. rer. nat. Nils Beckmann nils.beckmann@th-owl.de
Sprache	Deutsch
Curriculum	Elektrotechnik (B.Sc.) Wahlpflichtmodul Technische Informatik (B.Sc.) Wahlpflichtmodul MGT (B.Sc.) Wahlpflichtmodul Data Science (B.Sc.) (Wahl-)Pflichtmodul (je nach PO)
Lehrform	Vorlesung (2 SWS) Praktikum (2 SWS)
Zeitbedarf	60h Präsenz 90h Eigenstudium
Kreditpunkte	5 Punkte
Voraussetzungen	Formal: <i>keine</i> Inhaltlich: Programmiersprachen 1/2
Prüfung	Klausur, 90min, ohne Hilfsmittel

Literatur

Bitte beachten Sie die Datei „Literatur.pdf“ im ILIAS-Ordner „Zusatzfolien“ mit Literaturhinweisen.

- Teilweise ist die Literatur über die Hochschulbibliothek erhältlich, auch Online.
- Nutzen Sie die Literatur begleitend um Ihr Verständnis der Inhalte zu verbessern und zu vertiefen.

Themenübersicht

0	Einführung	Was ist Software?
1	Prozesse	Motivation, Beispielprozesse, ...
2	Anforderungen	Arten von / Kriterien für Anf., ...
3	Aufgaben	Tickets, ...
4	Spezifikation	Lastenheft & Pflichtenheft, ...
5	Entwerfen	Grob & Fein, Entwurfsmuster, ...
6	Vorgehensmodelle	Rollen, Methoden, Aktivitäten, ...
	• 7a Klassisch	u.a. Wasserfall, V-Modell, ...
	• 7b Agil	u.a. Scrum, XP, ...
	• 7c Generisch	u.a. Kanban, Spiralmodell, ...
8	Testen	Qualitätssicherung, V&V, ...
9	Evolution	Altsysteme, Varianten, Builds, ...
10	Metriken	Ressourcen, Ablaufplanung, ...
11	Regelwerke	Agiles Manifest, Lean, ...
12	Perspektiven	Reale Projektabläufe, Teams, ...

Vorlesungen und Praktika

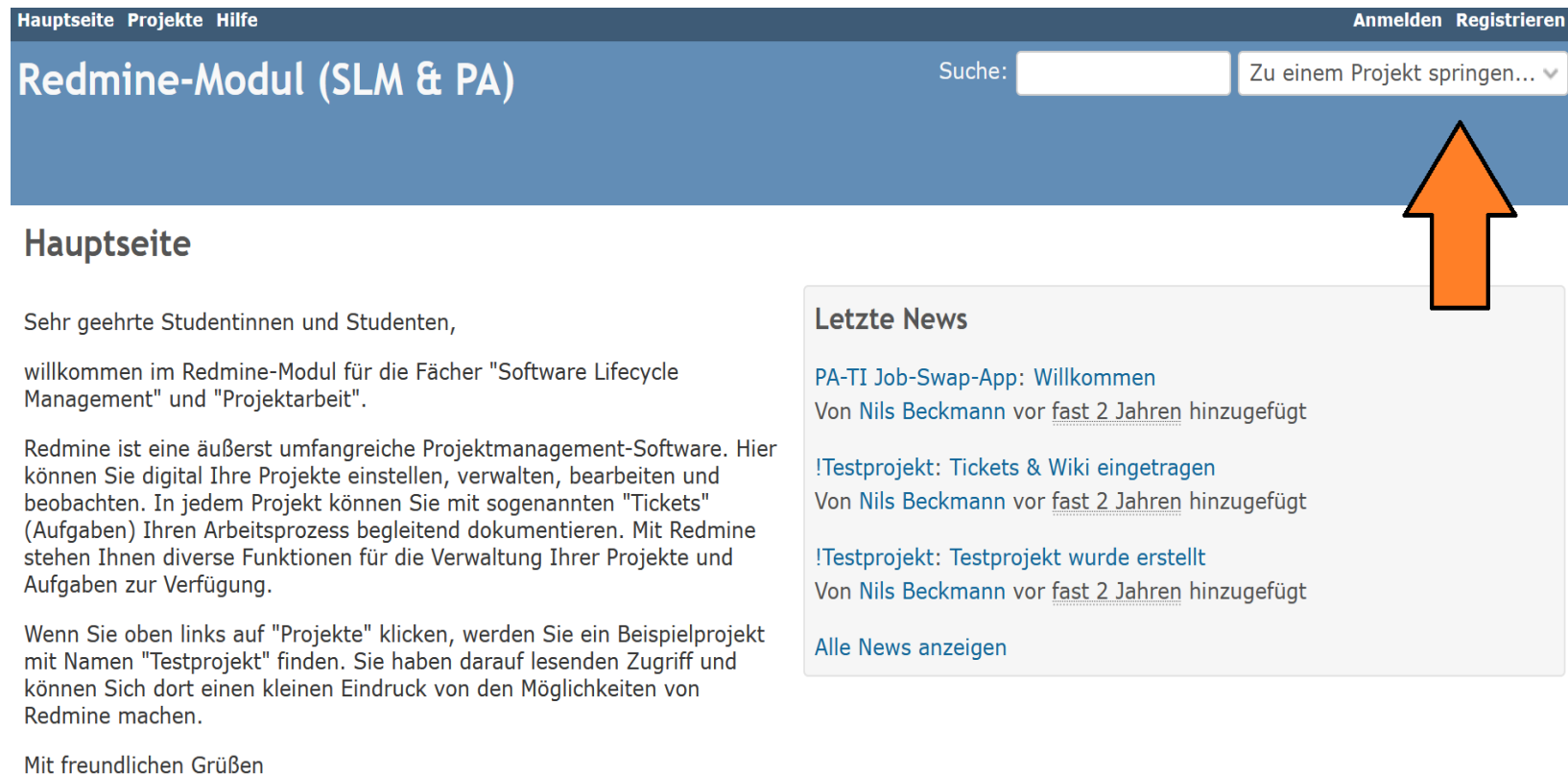
- Die **Vorlesungsnummer** passt inhaltlich immer zum jeweiligen Übungszettel/Praktikum (nur zu Praktikum 007 gibt es die drei Vorlesungen 107a, 107b, 107c).
- Jede Woche ist ein Übungszettel zu bearbeiten. Bei den **ÜZ 1/2/3** sollten die Teilnehmer einigermaßen synchron wochenweise arbeiten, damit ein Austausch möglich ist.
 - Bitte jeweilige **ÜA1 vor dem Praktikum bearbeiten!**
- Ab späteren Übungszetteln ab ÜZ 4 kann individueller gearbeitet werden in den einzelnen **Projektteams**.
- Die **aktive Teilnahme** am Modul wird durch Teilnahme in einem Teamprojekt und die Erstellung von entsprechenden **Projektergebnissen** dargestellt.

Ablauf der Praktika

- Praktika 1+2: Trainingsphase
 - Allgemeiner Einstieg in die Denkweise und Werkzeuge des Moduls
- Ab Praktikum 3: **Projektphase**
 - P3/P4: Projektideen und -initiierung ("Kick off")
 - Anwendung des gelernten Wissens im Rahmen von selbsterstellten und selbstgelenkten Projekten
 - Jeder Teilnehmer nimmt an **einem Teamprojekt** teil und ist Beobachter in einem weiteren Projekt
 - Durchlauf des gesamten Software Lifecycles von der Idee bis zum Abschluss

Redmine-Plattform

- Redmine ist eine Projektmanagementplattform
- <http://werre.ee.hs-owl.de/> (VPN-Zugang notwendig!)
- Bitte registrieren Sie sich dort (Aufgabe in ÜZ1/ÜA7)



The screenshot shows the Redmine-Modul (SLM & PA) homepage. The top navigation bar includes links for 'Hauptseite', 'Projekte', and 'Hilfe', along with 'Anmelden' and 'Registrieren'. A search bar and a dropdown menu 'Zu einem Projekt springen...' are also present. The main content area is titled 'Hauptseite' and contains a welcome message for students, a description of Redmine's features, and a 'Letzte News' section with recent updates. An orange arrow points to the 'Zu einem Projekt springen...' dropdown menu.

Hauptseite Projekte Hilfe Anmelden Registrieren

Redmine-Modul (SLM & PA) Suche: Zu einem Projekt springen... ▾

Hauptseite

Sehr geehrte Studentinnen und Studenten,

willkommen im Redmine-Modul für die Fächer "Software Lifecycle Management" und "Projektarbeit".

Redmine ist eine äußerst umfangreiche Projektmanagement-Software. Hier können Sie digital Ihre Projekte einstellen, verwalten, bearbeiten und beobachten. In jedem Projekt können Sie mit sogenannten "Tickets" (Aufgaben) Ihren Arbeitsprozess begleitend dokumentieren. Mit Redmine stehen Ihnen diverse Funktionen für die Verwaltung Ihrer Projekte und Aufgaben zur Verfügung.

Wenn Sie oben links auf "Projekte" klicken, werden Sie ein Beispielprojekt mit Namen "Testprojekt" finden. Sie haben darauf lesenden Zugriff und können sich dort einen kleinen Eindruck von den Möglichkeiten von Redmine machen.

Mit freundlichen Grüßen

Letzte News

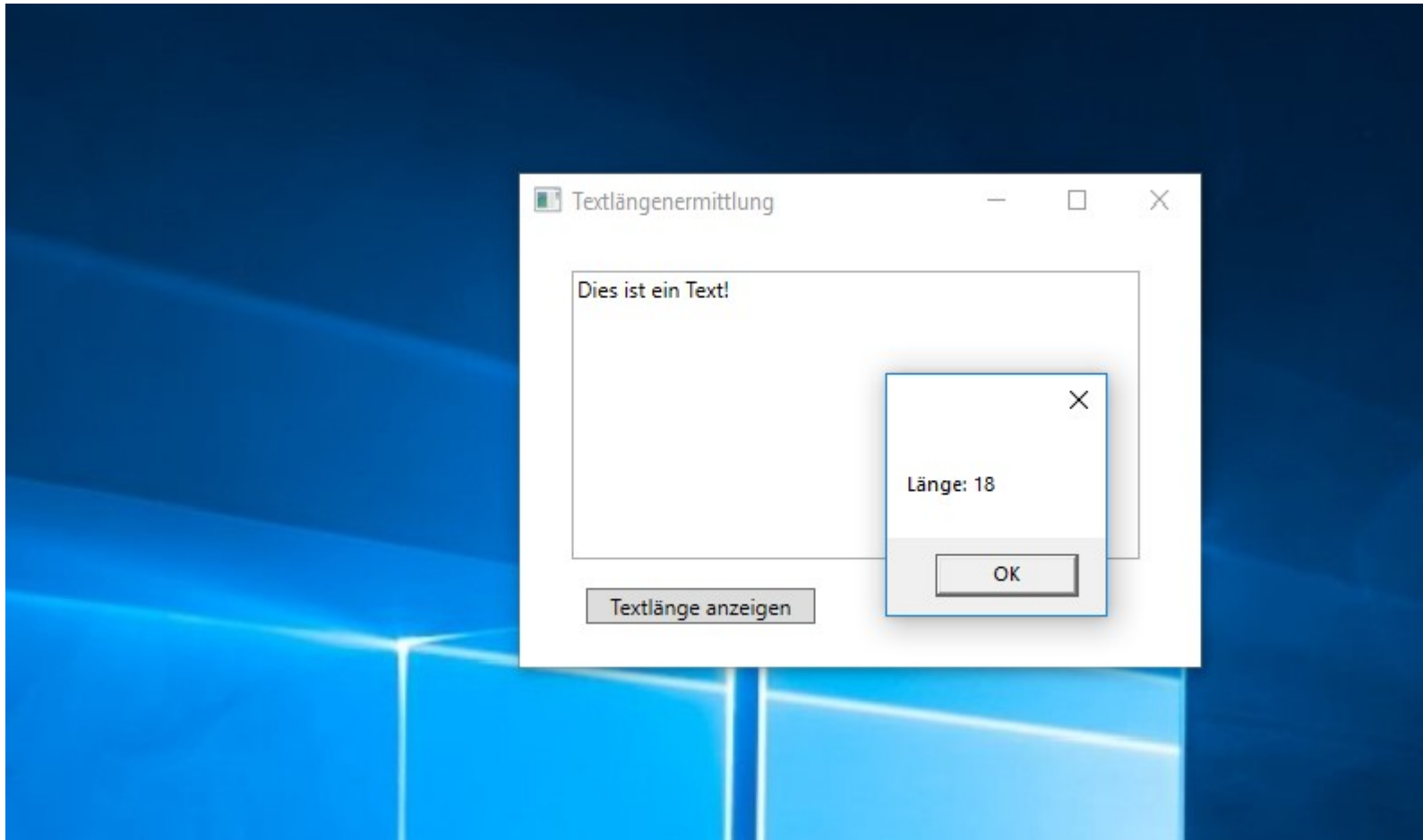
[PA-TI Job-Swap-App: Willkommen](#)
Von Nils Beckmann vor fast 2 Jahren hinzugefügt

[!Testprojekt: Tickets & Wiki eingetragen](#)
Von Nils Beckmann vor fast 2 Jahren hinzugefügt

[!Testprojekt: Testprojekt wurde erstellt](#)
Von Nils Beckmann vor fast 2 Jahren hinzugefügt

[Alle News anzeigen](#)

Was ist Software?



Ist das Alles?

Aber was ist mit diesen Dingen:

- A1: Das Ziel ist ein Programm zur Ermittlung der Länge eines Textes.
- A2: Die Länge des Textes soll durch Klicken auf einen Button angezeigt werden.
- A3: Wenn kein Text eingetragen ist, soll eine Warnmeldung erscheinen.
- A4: Die Länge des eingegebenen Textes soll maximal 20 Zeichen betragen

Das sind **Anforderungen** an die Software.

Und diesen:

- T1:
Schritte → Kein Text eingeben, Button drücken
Erwartetes Ergebnis → Warnmeldung
- T2:
Schritte → Wort "Hallo" eingeben, Button drücken
Erwartetes Ergebnis → Anzeige der Textlänge "5"
- T3:
Schritte → Eingabe von "Viel, viel zu langer Text",
Button drücken
Erwartetes Ergebnis → Warnmeldung

Das sind **Testfälle** für die Software.

Und diesen Fragen:

- Wer definiert die **Anforderungen**?
 - Der Kunde? Vielleicht werden unmöglich umsetzbare Anforderungen definiert!
Der Auftragnehmer? Vielleicht werden Kundenwünsche ignoriert!
- Wer schreibt die Software (macht die **Software-Entwicklung**)?
 - Wie lange wird/darf/soll dies dauern?
 - Falls es mehrere Entwickler sind: Welche Aufgabenpakete (AP) gibt es? Wer übernimmt welches AP? Wer achtet darauf, dass die richtigen AP's bearbeitet werden – dass niemand in eine Sackgasse gerät, dass auf Fristen geachtet wird, dass wichtige Kommunikation auch stattfindet?
- Wer testet die Software (gemäß den **Testfällen**) – wer definiert die Testfälle?
 - Wer korrigiert die Software und wann, falls Testfälle negativ ausfallen? Wie sollen die Korrekturen aussehen?
- Wer prüft, dass die Software allen Anforderungen entspricht (**Qualitätsprüfung**)?
 - Wer verbessert die Software und wann, falls Anforderungen nicht erfüllt sind?
- Wer liefert die Software zum Kunden aus und wann (**Auslieferung**)?

Gibt es dabei Dinge zu beachten: Weite Anreise, notwendige Hardwarekomponenten, bestimmte Ansprechpartner vor Ort müssen verfügbar sein, Produktionsstart ist terminlich an die Softwaregekoppelt, usw.

Und diesen Fragen:

- Woher weiß der Kunde, wie die Software zu verwenden ist?
 - Wer schreibt eine passende **Dokumentation**? Wird dieses (oder ein anderes) Entwicklungsteam die Software auch noch in einem Jahr verstehen (und weiterentwickeln können)? Ist die Software erweiterbar, rückwärtskompatibel, lesbar?
- Woher wissen andere potentielle Kunden von diesem Softwareprodukt?
 - Auf welchen Kanälen wird über das Softwareprodukt berichtet und informiert? Welche Firmen könnten diese Software auch benötigen?
- Wieviel wird die **Entwicklung kosten**?
 - Wieviel Personalkosten wird es geben? Wie viele Entwickler brauchen wir und wie lange? Welche Hardwarekomponenten müssen gekauft werden? Müssen Räume gemietet werden?
- Wie haben sich die **Projektmitarbeiter** durch das Projekt weiterentwickelt?
 - Welche Fähigkeiten haben sie dadurch dazuerlangt? Wie ist die Stimmung im Team? Welche ethischen Verhaltensweisen? Welcher Mitarbeiter hat wo Schwächen oder wo besondere Stärken (schnell, effektiv, einsatzfreudig)? Wie wurde kommuniziert? Sind alle Informationen vollständig und rechtzeitig übergeben worden?

Software ist mehr

Der gesamte
Softwareentwicklungsprozess
umfasst weit mehr als nur das
Computerprogramm.

Defintionen: "Software"

Menge von Programmen oder Daten zusammen mit begleitenden Dokumenten, die für ihre Anwendung notwendig oder hilfreich sind.

[Hesse et al, 1984]

Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.

[IEEE 610.12 - IEEE Standard Glossary of Software Engineering Terminology, 1990]

Ergebnisse der Softwareentwicklung

- Anforderungsspezifikation ("Requirements")
- Anwendungsfälle ("Use-Cases")
- GUI-Prototyp
- (Software-)Architekturdokumente (UML, Entwurfsmuster, usw.)
- Projektablaufpläne ("Gantt"-Balkendiagramme, Rollen-/Aufgabenverteilungen, ...)
- Quelltext (Source Code + Kommentare; Versionsverwaltung)
- Testplan, Testbericht, Fehlerberichte ("Bugs")
- (Software-)Dokumentation
- Installierte (ausgelieferte) Anwendung
- Archivierung (Versionen und Dokumente/Aufträge/Rechnungen/...)

siehe z.B. Plewan, Poensgen, „Produktive Software-Entwicklung“

Kategorisierung von Software

- **Umgebung:** PC-Programme (Windows, Mac, Linux)
oder Embedded/Firmware (fest in einem Gerät eingebaute Software)
oder Cloud/Server (für Web-Anwendungen), ...
- **Lizenz:** Kommerziell,
Freeware,
Shareware (zur Testnutzung kostenlos),
Adware (Einblendung von Werbung), ...
- **Bestimmte Eigenschaften:** Realtime oder Non-Realtime,
Verteilt oder Monolithisch (gebunden an bestimmte Bibliotheken, Hardware, etc.),
Ressourcen-Schonend oder -Intensiv,
Portabel oder Nicht-Portabel (Abhängigkeit von einer Plattform),
Seriell oder Parallel (auf mehreren Cores), ...
- **Verbreitung:** Standard (Kunde erwirbt fertiges Software-Produkt)
oder Individuell (Kunde stimmt Software mit ab)
- **Quelltext-Verfügbarkeit:** Open Source oder Proprietär
oder Freeware (Quelltext bleibt urheberrechtlich geschützt)
- **Nutzer:** Private Nutzer (B2C), Firmen (B2B), Staatliche Organe, ...
- usw.

Zusätzliche Herausforderungen bei Software-Entwicklung in Realität

- Mitarbeiter haben **keine Übersicht**, geben falsche Hinweise und machen Fehler
- Unternehmen können **zukünftige Entwicklungen**/Möglichkeiten nicht richtig einschätzen
- Es wird das **falsche Problem** angegangen oder die vermeintliche Lösung wird verkompliziert
- **Zeitpläne** werden nicht eingehalten, unvorhergesehene Aufgaben treten auf, Fehler müssen korrigiert werden
- **Wichtige Informationen** werden nicht kommuniziert, manches wird mehrfach besprochen oder mehrfach bearbeitet
- usw.

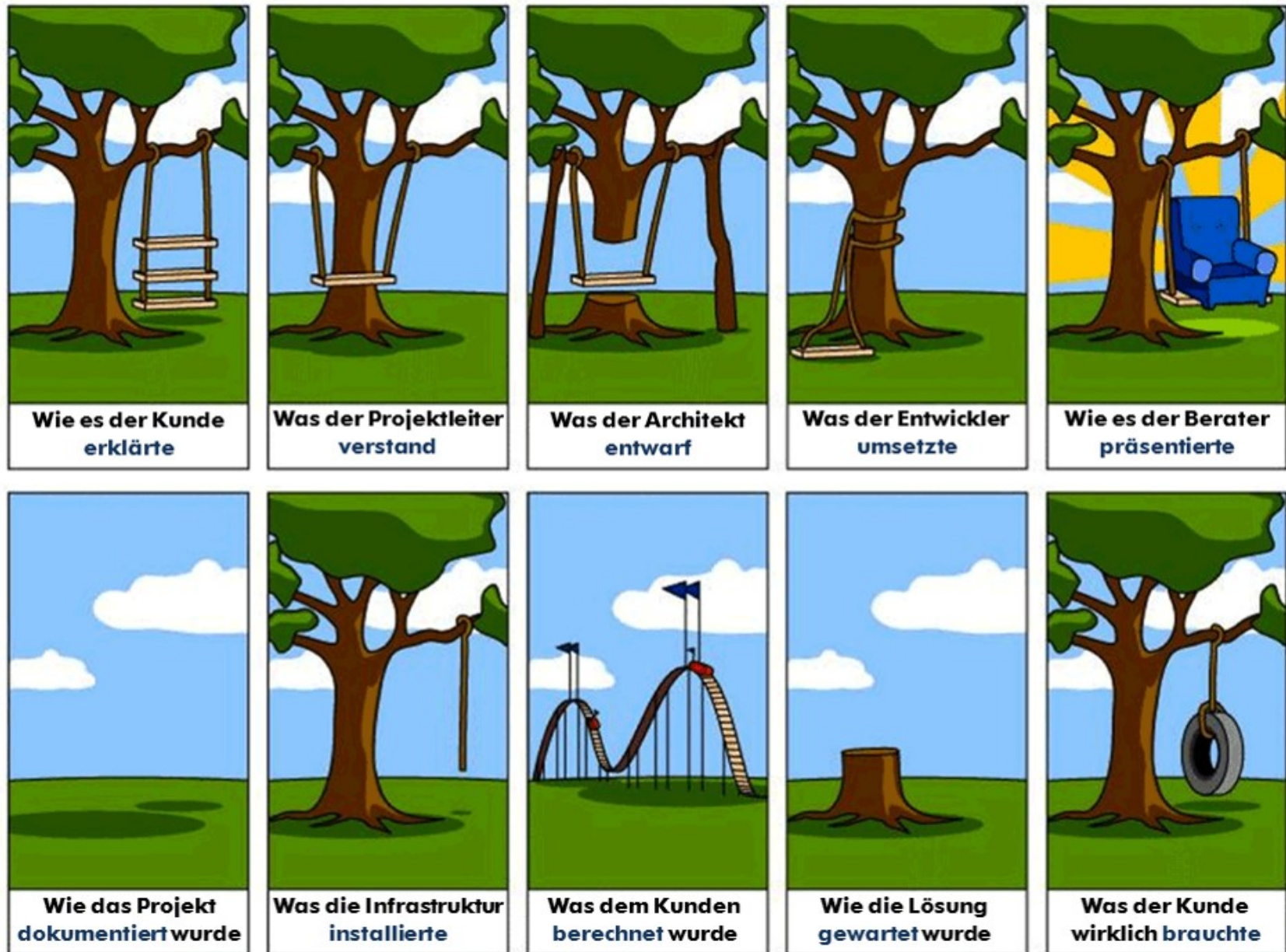
Anforderungen im Kundenkontakt

- Ansprechpartner "**denken**" **nicht in Software**: Keine Programmierer, nicht Computer-affin
- Kunden/Investoren kennen ihre **wirklichen Wünsche** nicht oder können sie nicht ausdrücken

„One of the keys to a successful software project is to build the system according to users' needs, but unfortunately, many software companies fail to gather the right requirements from stakeholders, mostly due to stakeholders fail to express their needs and concerns clearly and thoroughly.“

[VisualParadigm-UExceler]

...und das kann dazu führen:

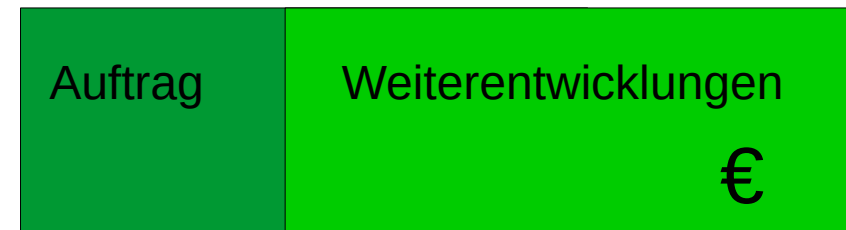


Wirtschaftliche Aspekte

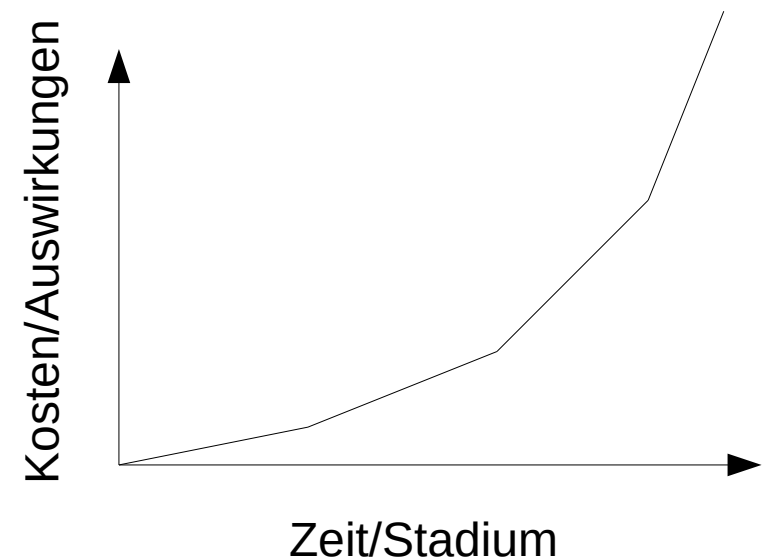
- Etwa 60% der Kosten der Software-Entwicklung sind Entwicklungskosten und weitere 40% sind Kosten für Tests



- Bei Auftragssoftware übersteigen die Kosten für Weiterentwicklungen oft die Kosten für den eigentlichen Auftrag



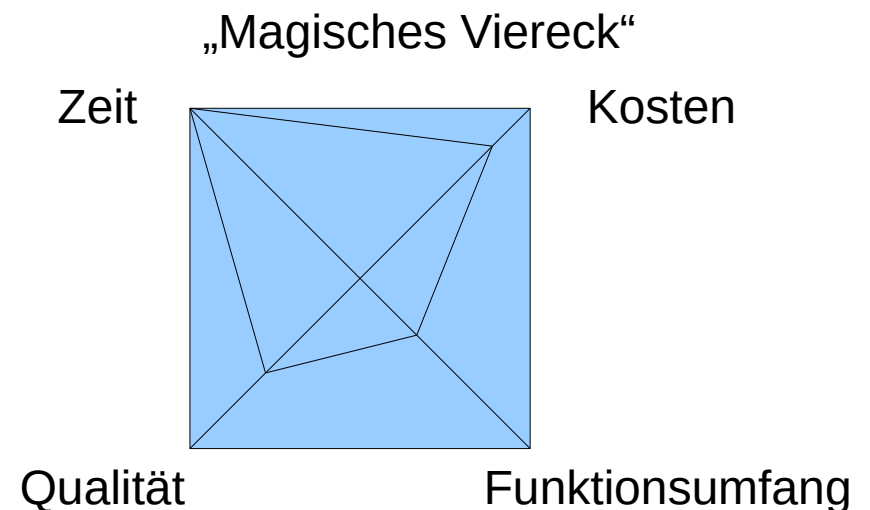
- Die Kosten und Auswirkungen von Fehlern nehmen exponentiell mit dem Entwicklungsstadium zu
- Bei ca. 10-100 Zeilen Quelltext pro Entwickler und Tag (pro Tag etwa 200-500€) also 2-50€ pro Zeile



[Betriebssysteme 5-50 Mio. Zeilen => min. 10 Mio. €]

Probleme bei der Software-Entwicklung

- **Qualität:** unzuverlässig, benutzerunfreundlich, schlecht/nicht dokumentiert, inkompatibel, fehlerhaft, nicht-übertragbar
- **Funktionsumfang:** Gewünschte Funktionen werden von der Software nur zum Teil oder gar nicht erfüllt
- **Kosten:** Überziehung des geplanten Budgets für die Entwicklung
- **Zeit:** Überschreitung des Fertigstellungstermins, Verzögerungen innerhalb des Projektplans, ungeplante Wartezeiten



Realweltliche Bedeutung und Probleme von Software

- 4. Juni 1996: **Absturz unbekannte Rakete** ("Ariane 5")
Fehler: Konvertierung von 8-Byte-Double nach 2-Byte-Integer, Zahl war leider > 32768 ...
Ca. 500 Mio. € für Satellit+Rakete, vorher ca. 7 Mrd. € Entwicklungskosten
- 24. Juli 2013: Student findet **Zahlendreher beim Scannen von Dokumenten** mit Xerox-Scannern
Fehler: Das Bild-Kompressionsverfahren ersetzte ähnlich aussehende Teilbilder durch Kopien
Weltweit stehen hunderttausende Xerox-Scanner/Drucker in Arztpraxen, Kanzleien, Architekten- und Ingenieurbüros, usw.
- 25. Februar 1991: **Raketenabwehrsystem berechnet Raketenflugbahn falsch**
Fehler: Systemzeit war auf Grund der Laufzeit von über 100h und Rundungsfehler um 1/3 Sekunde fehlerhaft, bei Raketengeschwindigkeit ca. 1,7 km/s Verfehlung um ca. 1/2 Kilometer, Gebäude zerstört
- 31. Oktober 1993: **Eigentlicher Eröffnungstermin** Flughafen Denver
Fehler: Neues Gepächsystem mit 300 Computern durch Ethernet-Überlastung ausgefallen
Zerquetschte Koffer, Sortierung von Hand, Eröffnung um 9 Monate verschoben, Probleme hielten bis 2005 an, mehrere Mrd. € Verluste
- 29. Oktober 2018: **Fehlerhafte Software** MCAS führt zu zwei Flugzeugabstürzen
Fehler: Software erhielt falsche Anstellwinkel und korrigierte mit unnötigen Manövern. Die Piloten wurden mangelhaft informiert über die Eingriffsmöglichkeiten von MCAS.
- uvm.

Quelltextzeilen Projekte

Anwendung	Quelltextzeilen (ca.)
• "Hallo Welt"-Anwendung	1-10
• Kopfrechentrainer (siehe PS1)	10-100
• Ortsverzeichnis (siehe PS1)	100-500
• Spiel "Tic Tac Toe" mit GUI	500-1.000
• Messenger "Signal" [1]	5.000-10.000
• Bildbearbeitungsprogramm "Gimp" [2]	500.000-1.000.000
• Betriebssysteme	5.000.000-50.000.000

[1] Signal-Repository (open source), <https://github.com/signalapp/Signal-Android>

[2] Gimp-Repository (open source), <https://www.openhub.net/p/gimp>

Fehlerquotient in Software

Wie viele Fehler pro 1.000 Programmzeilen?

- | | | |
|---|--------|--------------------|
| • Anspruch Krankenhaus, Weltraum, Militär | < 0,5 | ("stabil", "reif") |
| • Anstrebenswert/Akzeptabel | < 1-10 | |
| • Kann gerichtlich problematisch sein | > 10 | ("fehleranfällig") |
| • Typische kommerzielle Software | 5-50 | |

Würden Sie Sich in ein Flugzeug setzen,
dessen Software Sie geschrieben haben?

Software-Entwicklung braucht Software-Management...



...aber Strukturiertes Software-Management!

- hinsichtlich Verteilung von Aufgaben
- hinsichtlich Verteilung von Rollen

Grundfrage: WER macht WAS? (Bis WANN?)

Typische Rollenstrukturen

Management

„Der Blick aufs Ganze“
Welche Produkte?
Welche Ziele?

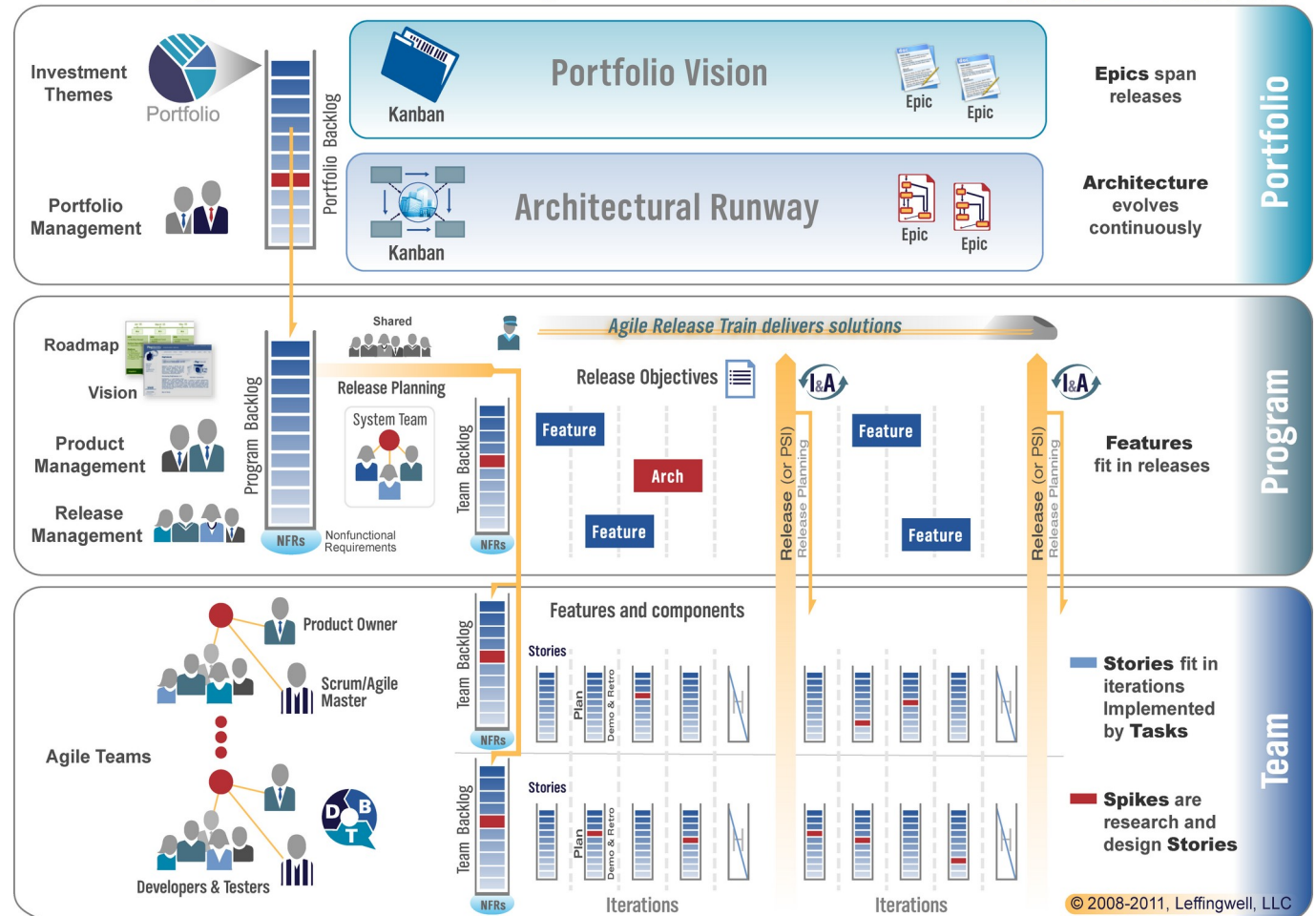
Organisation

„Der praktische Ansatz“
Mit welchen Ressourcen
(Zeit, Geld, Mitarbeiter,
Räume, Geräte) das Ziel
erreichen?

Entwicklung

„Detailarbeit“
Wie löse ich das Problem?
Wie mache ich das?

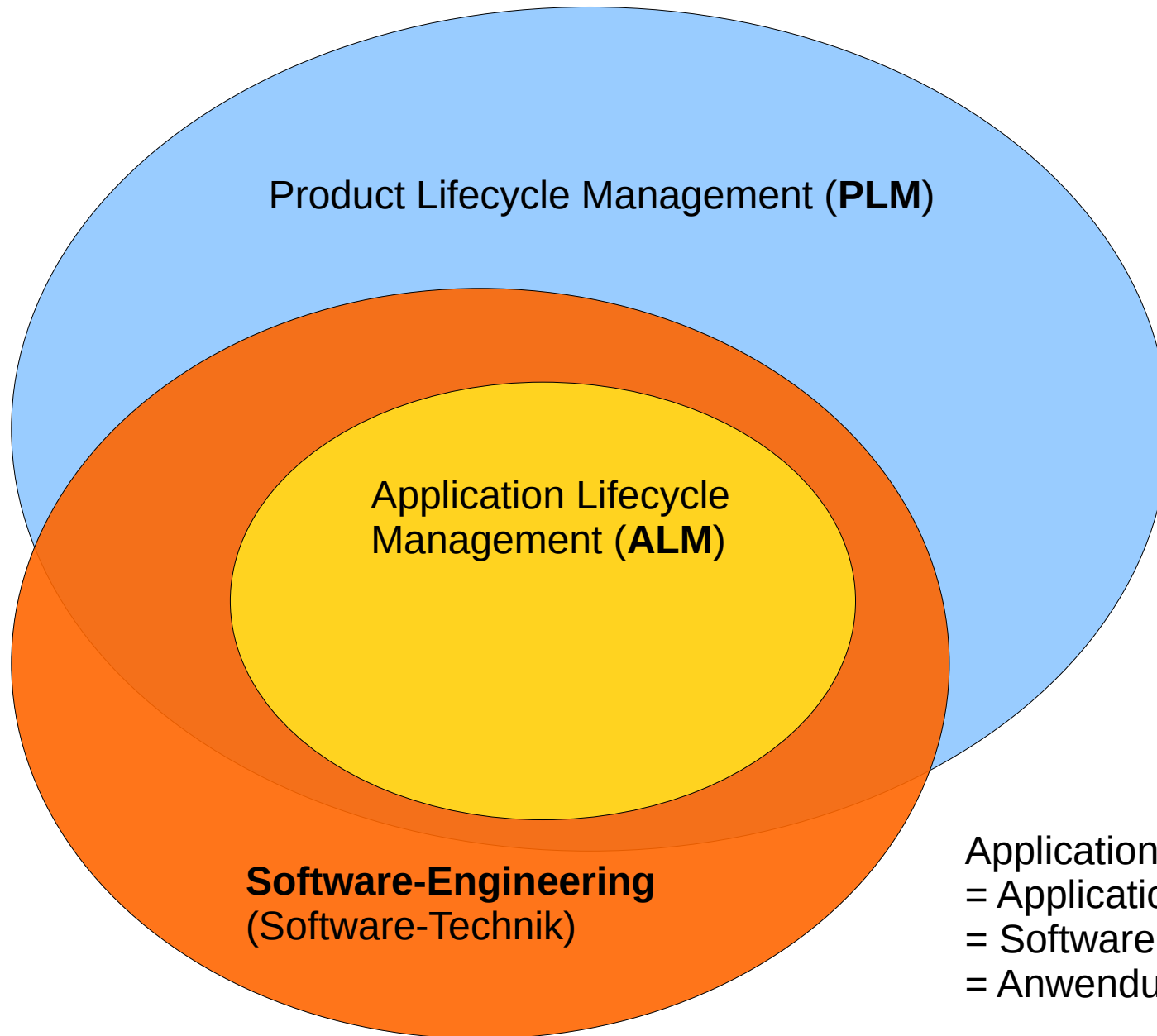
Scaled Agile Framework™ Big Picture



See also www.scalingsoftwareagility.wordpress.com and Leffingwell, D. *Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise*, Addison-Wesley (Pub. 2011) v 0.8

Auch bei Ein-Personen-Software-Projekten gilt es verschiedene Rollen einzunehmen!

Begriffliche Abgrenzung



ALM beinhaltet:

+ Planung
+ Entwicklung
+ Betrieb
+ Betreuung
...von Software!

Application Lifecycle Management (ALM)
= Application Management (AM)
= Software Lifecycle Management
= Anwendungsmanagement (AM)

Begriffliche Definitionen

- **PLM** (aus der Hardware-Entwicklung): "Gesamtheitliche Organisation des Produktionslebenszyklus auf Basis methodischer und organisatorischer Maßnahmen und Anwendung der Informationstechnologie"
 - Geschichtlich entwickelt aus:
 - EDM (Engineering Data Management)
 - PDM (Product Data Management)
- **ALM** (aus der Software-Entwicklung): "Koordination Softwareentwicklungsaktivitäten sowie das Management von Softwareartefakten, zum Beispiel von Anforderungen und Testfällen"

Modulbeschreibung der Inhalte

Kompetenzen:

Die Studierenden kennen die wesentlichen Elemente des Application Lifecycle Managements (ALM). Sie können die Gesamtstruktur von Softwareprodukten und deren Lifecycle erstellen, beschreiben und analysieren. Softwareprodukte werden dabei sowohl von der technischen als auch von der Produkt-Seite aus betrachtet. Das Zusammenspiel und die Abhängigkeiten der Methoden und Werkzeuge sind bekannt. Die Studierenden kennen Prozesse, Methoden und Werkzeuge des ALM und können sie anwenden und bewerten.

Inhalt:

Vorlesung:

Grundkonzepte, Produktmanagement, Requirements Engineering, Defect Tracking, Organisationsstrukturen von Softwareteams, Management und Vertrieb, Entwicklungsteam und Projektleitung

Praktikum:

Die in der Vorlesung vorgestellten Themen und Vorgehensweisen sowie Werkzeuge werden in einem Beispielprojekt in einen Zusammenhang gebracht. Die Implementierungen werden vom Dozenten mit den Studenten diskutiert, aber nicht benotet.

Überblick und Inhalte von ALM-Vorlesung und -Praktikum

- **Software-Entwicklung**

- ... Aspekte > Definition, Wirtschaftlichkeit, Bedeutung, Ethik, ...
- ... Prozesse > Lebenszyklus, Phasen, ...
- ... Vorgehensmodelle > V-Modell, Scrum, Wasserfall, Spiralmodell, XP, ...
- ... Philosophie > Kanban, Lean, Prototyping, ...

- **CASE ("Computer Aided Software Engineering")**

Editoren, IDE's, Compiler, (Enterprise-)Wikis, Ticket-Systeme, Prozess-Software, Versionsverwaltung, ...

- **Software-Management-Disziplinen**

- Requirements Management (REQ) > Anforderungen, Lasten-/Pflichtenheft
- Test and Quality Management (TQM) > Testen, Qualitätssicherung
- Issue and Defect Management (IDM) > Umwelt, Bugs, Probleme
- Change and Configuration Management (CCM) > Neuerungen einführen
- Audits, Metrics, Reports (AMR) > Quantifizierung, Berichte
- Resource Management (RM) > Mitarbeiter, Zeit, Geld, Planung
- Build and Release Management (BRM) > Projektabschluss, Auslieferung
- Variant Management (VM) > Versionen, Varianten, Zusatzmodule
- etc.

Inhalte von ALM: Planung, Entwicklung, Betrieb, Betreuung von Software

Methoden und Werkzeuge des ALM

- **Methode:**

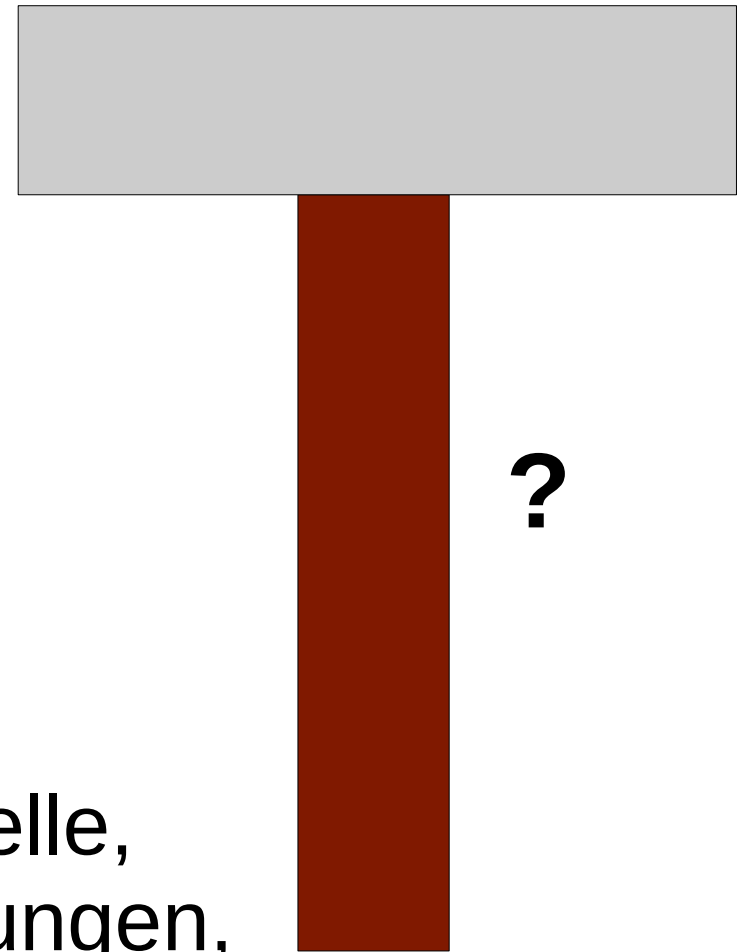
- Planmäßiger, formalisierter Ablauf/Verfahren, welches für viele für die Methode geeignete Probleme zu einer sinnvollen Lösung führt.
- Methoden können entsprechende Werkzeuge nutzen.

- **Beispiel:**

- Methode: Erstellung eines Lastenhefts
- Werkzeug: Lastenheft

Methoden und Werkzeuge des ALM

- Werkzeug?
- Mentale Arbeit nutzt mentale Werkzeuge!



- Prozesse, (Vorgehens-)Modelle, Rollen, Diagramme, Vorstellungen, Konzepte, Richtlinien, usw.

Zusammenfassung

- **Software ist mehr als nur das Computerprogramm**

Menge von Programmen oder Daten zusammen mit begleitenden Dokumenten, die für ihre Anwendung notwendig oder hilfreich sind. [Hesse et al, 1984]

- **Software-Entwicklung ist mehr als nur Programmieren**

"Application Lifecycle Management (ALM)" begleitet und strukturiert den Softwarelebenszyklus durch Werkzeuge und Methoden – moderne Software-Entwicklung nutzt ALM.

Die Blickwinkel auf ALM



Variant Management (VM)



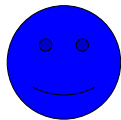
Issue and Defect Management (IDM)



Configuration And Change Management (CCM)



Requirements Management (REQ)



Resource Management (RM)



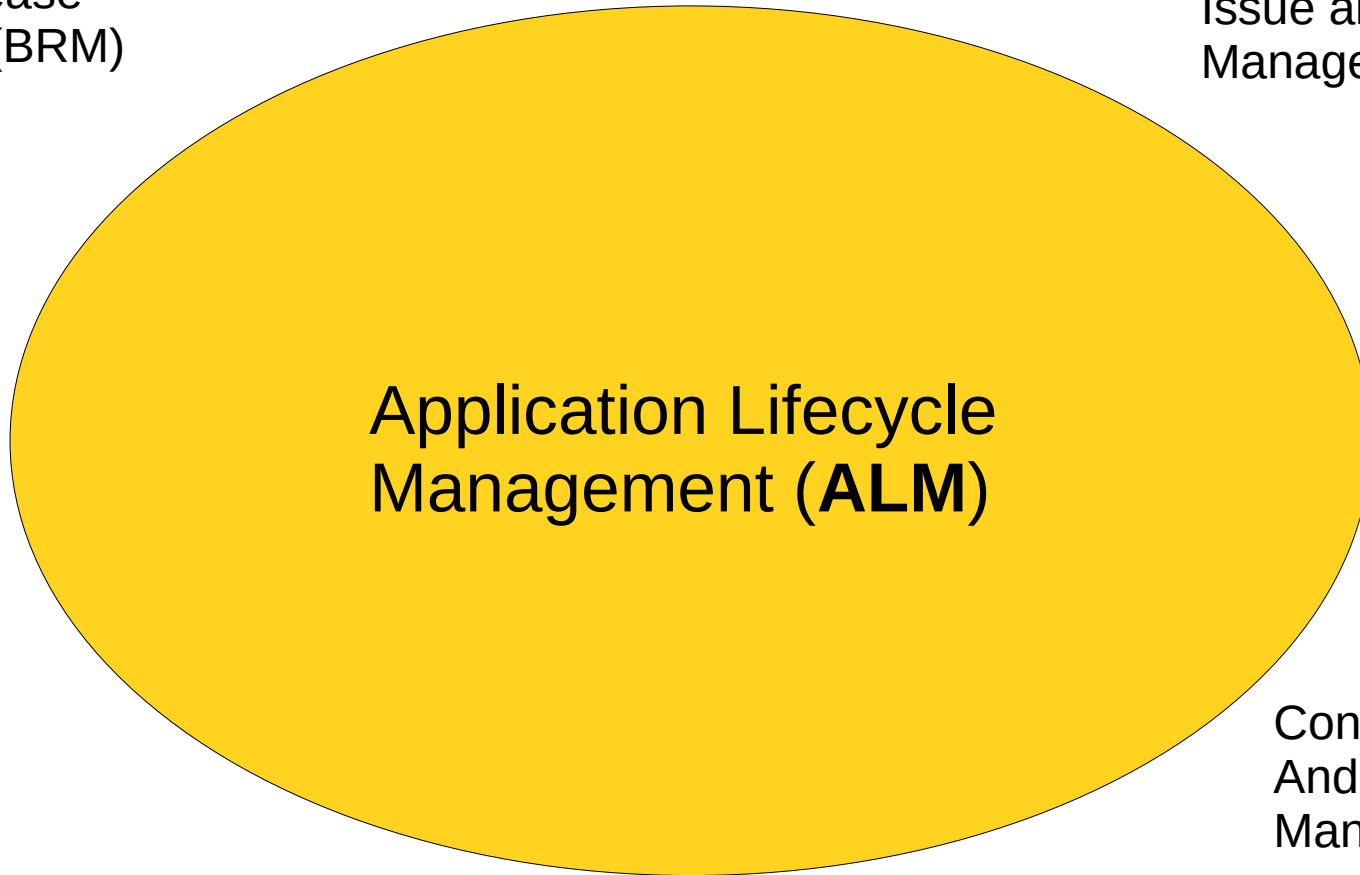
Test and Quality Management (TQM)



Metrics, Audits, Reports (AMR)



Build and Release Management (BRM)



Fragen?